TRƯỜNG ĐẠI HỌC .....

KHOA .....

BÁO CÁO BÀI TẬP LỚN

MÔN: AN TOÀN THÔNG TIN

Đề tài: Mô phỏng tấn công DDoS bằng botnet, triển khai phòng chống và cân bằng tải trên môi trường ảo hóa

Nội dung	Thông tin
Sinh viên thực hiện	.....
Mã số sinh viên	.....
Lớp	.....
Giảng viên hướng dẫn	.....
Thời gian thực hiện	Tháng 11 năm 2026

Mục lục

1. Mở đầu
2. Cơ sở lý thuyết
3. Thiết kế và triển khai hệ thống lab
4. Kết quả thực nghiệm
5. Thảo luận
6. Kết luận
7. Tài liệu tham khảo
8. Phụ lục

---

1. Mở đầu

1.1. Lý do chọn đề tài

Tấn công từ chối dịch vụ phân tán (DDoS) là một trong những mối đe dọa phổ biến nhất đối với hệ thống thông tin. Khi nhiều máy tính bị nhiễm mã độc cùng gửi lưu lượng đến một mục tiêu, dịch vụ có thể bị quá tải và ngừng đáp ứng người dùng hợp lệ. Để hiểu rõ cơ chế tấn công cũng như cách phòng chống, việc dựng một mô hình thu nhỏ có đầy đủ ba thành phần — botnet tấn công, hệ thống phòng thủ và cơ chế cân bằng tải — là cần thiết.

Bài tập lớn được thực hiện theo ba bước:

1. Sử dụng botnet để tự tấn công hai máy tính lẫn nhau trong môi trường lab.
2. Triển khai các công cụ phòng chống mã nguồn mở: Suricata, tường lửa UFW và giới hạn tần suất của Nginx.
3. Triển khai cân bằng tải và kiểm chứng khả năng chịu lỗi khi một backend ngừng hoạt động.

1.2. Mục tiêu

- Xây dựng môi trường ảo hóa 5 máy ảo trên VirtualBox, điều khiển hoàn toàn từ WSL.
- Mô phỏng kiến trúc botnet gồm máy chủ C2 và hai bot agent, thực hiện tấn công có kiểm soát vào máy nạn nhân.
- Triển khai phòng thủ nhiều lớp: phát hiện bằng chữ ký, chặn tự động bằng tường lửa và giới hạn tải ở tầng ứng dụng.
- Triển khai Nginx làm bộ cân bằng tải cho hai backend, kiểm chứng phân phối round-robin và cơ chế failover.

1.3. Phạm vi, giới hạn và yếu tố an toàn

Toàn bộ thực nghiệm chỉ diễn ra trong mạng NAT nội bộ `10.10.10.0/24` của VirtualBox, không có lưu lượng nào đi ra Internet và không tấn công bất kỳ hệ thống bên ngoài nào.

Vì lý do an toàn và pháp lý, báo cáo **không tải và không sử dụng botnet thật**. Thay vào đó, nhóm mô phỏng lại kiến trúc và kỹ thuật của các công cụ tấn công mã nguồn mở như LOIC/Hping3 bằng các script Python ngắn, chỉ có chức năng kết nối C2, nhận lệnh và sinh lưu lượng đến đúng máy nạn nhân trong lab. Các script không có chức năng lây lan, không có cơ chế tồn tại (persistence) và mỗi đợt tấn công bị giới hạn thời gian (25 giây).

Các giới hạn của mô hình:

- Quy mô nhỏ (2 bot, 30 luồng mỗi bot) nên không thể làm sập hoàn toàn một máy chủ, mục tiêu là quan sát cơ chế và đo đạc.
- Toàn bộ máy ảo chạy trên một máy vật lý, tài nguyên CPU/RAM bị chia sẻ.
- Địa chỉ IP nguồn của bot là IP thật trong lab, chưa mô phỏng giả mạo IP nguồn (IP spoofing).

---

2. Cơ sở lý thuyết

2.1. Tấn công DDoS

DDoS là hình thức tấn công làm cạn kiệt tài nguyên của mục tiêu (băng thông, kết nối, CPU, bộ nhớ) bằng lưu lượng đến từ nhiều nguồn phân tán. Có ba nhóm chính:

- **Tấn công thể tích (volumetric):** UDP flood, ICMP flood, amplification — làm đầy băng thông.
- **Tấn công giao thức:** SYN flood, khai thác bắt tay TCP để làm cạn bảng kết nối.
- **Tấn công tầng ứng dụng:** HTTP flood, slowloris — gửi nhiều yêu cầu hợp lệ về hình thức nhưng tiêu tốn tài nguyên xử lý.

Trong bài này, bot được cài ba chế độ `http`, `syn` và `udp`; kịch bản chính sử dụng HTTP flood vì đây là dạng tấn công tầng ứng dụng phổ biến và dễ quan sát qua access log.

2.2. Botnet và hạ tầng C2

Botnet là tập hợp các máy bị điều khiển từ xa (bot), nhận mệnh lệnh từ máy chủ Command and Control (C2). Kiến trúc tối thiểu gồm:

- **C2 server:** quản lý danh sách bot, phát lệnh tấn công.
- **Bot agent:** kết nối đến C2, chờ lệnh và thực thi.
- **Kênh điều khiển:** trong lab dùng TCP cổng 9999 cho bot và cổng 9998 chỉ lắng nghe trên loopback của C2 cho người điều hành.

Mô hình này giúp tách vai trò "ra lệnh" và "thực thi", đúng với cách botnet thật hoạt động, nhưng được giới hạn trong mạng lab.

2.3. Công cụ phòng chống

Suricata là hệ thống phát hiện/xâm nhập mã nguồn mở, phân tích lưu lượng theo chữ ký (signature) và các ngưỡng thống kê. Trong lab, Suricata chạy ở chế độ IDS trên card mạng `enp0s3`, ghi cảnh báo ra `fast.log`.

UFW (Uncomplicated Firewall) là lớp quản lý tường lửa đơn giản trên nền iptables/nftables. UFW được dùng để chặn IP nguồn đang tấn công và vẫn cho phép các cổng quản trị cần thiết.

Nginx đóng hai vai trò: cân bằng tải ở cổng 80 và giới hạn tải ở tầng ứng dụng bằng `limit_req` (số yêu cầu/giây theo IP) và `limit_conn` (số kết nối đồng thời theo IP).

Cơ chế chặn tự động: một tiến trình theo dõi `fast.log`, trích xuất IP nguồn từ cảnh báo và gọi `ufw insert 1 deny from <ip>`, nhờ đó rút ngắn thời gian phản ứng từ vài phút xuống còn 1–2 giây.

2.4. Cân bằng tải và chịu lỗi

Nginx dùng khối `upstream` để phân phối yêu cầu đến nhiều backend theo thuật toán round-robin mặc định. Hai tham số quan trọng:

- `max_fails=1 fail_timeout=5s`: sau một lần thất bại, backend bị đánh dấu "down" trong 5 giây.
- `proxy_next_upstream error timeout http_502 http_503 http_504`: nếu backend hiện tại lỗi, Nginx chuyển ngay yêu cầu sang backend còn lại.

Cơ chế này vừa tăng khả năng phục vụ vừa đảm bảo dịch vụ vẫn hoạt động khi một backend gặp sự cố.

---

3. Thiết kế và triển khai hệ thống lab

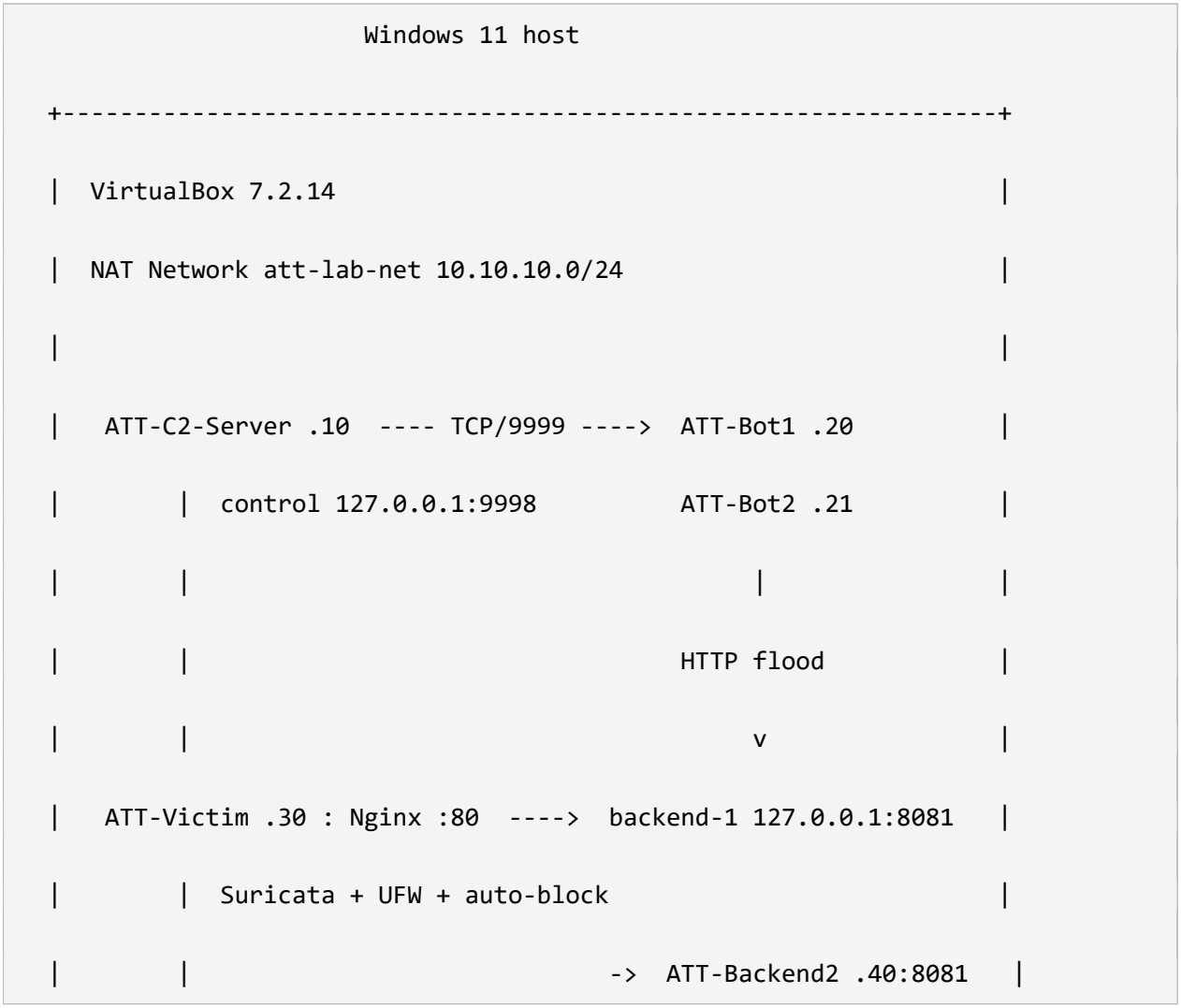
3.1. Môi trường

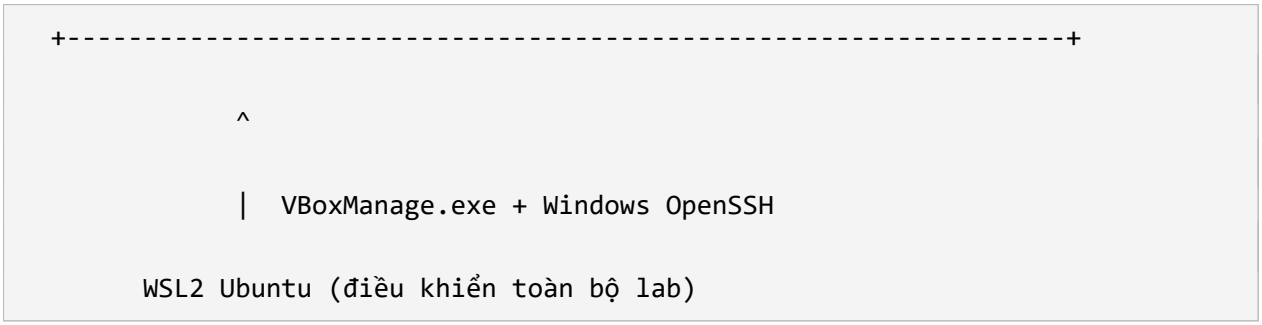
Thành phần	Thông số
Máy vật lý	Windows 11, VirtualBox 7.2.14
Hệ điều hành khách	Ubuntu Server 22.04 LTS

Thành phần	Thông số
Môi trường điều khiển	WSL2 Ubuntu, gọi `VBoxManage.exe` của Windows
Mạng lab	NAT Network `att-lab-net`, `10.10.10.0/24`, gateway `10.10.10.1`
Truy cập SSH	Port-forward 2222–2226 của host về cổng 22 của từng VM
Cấu hình tự động	cloud-init NoCloud (ISO nhãn `cidata`) + netplan

Do WSL2 có loopback tách biệt với Windows, các script không dùng `ssh` của WSL mà gọi `C:\Windows\System32\OpenSSH\ssh.exe`. Tiến trình SSH này chạy trên Windows nên `127.0.0.1:<port-forward>` trỏ đúng vào VirtualBox. Khóa riêng được siết quyền bằng `icacls` để Windows OpenSSH chấp nhận.

3.2. Kiến trúc mạng





VM	IP	Vai trò	Tài nguyên
ATT-C2-Server	10.10.10.10	C2, điều khiển bot	1 vCPU / 768 MB
ATT-Bot1	10.10.10.20	Bot agent	1 vCPU / 768 MB
ATT-Bot2	10.10.10.21	Bot agent	1 vCPU / 768 MB
ATT-Victim	10.10.10.30	Nginx LB, Suricata, UFW	2 vCPU / 1536 MB
ATT-Backend2	10.10.10.40	Backend Nginx :8081	1 vCPU / 768 MB

Mỗi VM có hai card mạng: card 1 gắn NAT Network để nhận IP tĩnh `10.10.10.x`, card 2 dùng NAT riêng để cài gói và cập nhật trong lúc khởi tạo.

3.3. Quy trình dựng máy ảo

Mỗi VM được tạo từ một VMDK gốc, sau đó gắn thêm ISO cloud-init chứa ba tệp:

- `user-data`: tạo người dùng, cài gói, ghi cấu hình dịch vụ và chạy script thiết lập.
- `meta-data`: đặt `instance-id` và hostname.
- `network-config`: cấu hình netplan, gán IP tĩnh theo địa chỉ MAC.

Một điểm quan trọng: netplan yêu cầu MAC ở dạng có dấu hai chấm (`08:00:27:AA:00:04`). Nếu dùng chuỗi 12 ký tự liền, netplan từ chối toàn bộ tệp và cả hai card mạng đều ở trạng thái DOWN.

Script `scripts/rebuild_cidata.sh` cho phép dựng lại ISO và khởi động lại VM khi cần thay đổi cấu hình, đảm bảo lab có thể tái lập nhiều lần.

3.4. Mô phỏng botnet

Máy chủ C2 lắng nghe cổng 9999 cho bot và cổng 9998 cho người điều hành. Lệnh điều khiển có dạng:

```
attack <ip> <port> [duration] [http|syn|udp]

bots
```

Đoạn mã phát lệnh tấn công:

```
def send_attack(target_ip, target_port, duration, attack_type):

    cmd = json.dumps({

        "cmd": "attack",

        "target_ip": target_ip,

        "target_port": target_port,

        "duration": duration,

        "type": attack_type,

    }) + "\n"

    with BOTS_LOCK:

        items = list(BOTS.items())

    sent = 0

    for bot_id, conn in items:

        try:

            conn.sendall(cmd.encode())

            sent += 1

        except OSError:

            with BOTS_LOCK:

                BOTS.pop(bot_id, None)
```

```
    log(f"ATTACK {target_ip}:{target_port} type={attack_type} "

        f"duration={duration}s -> {sent} bot(s)")

    return sent
```

Bot agent kết nối đến C2, gửi thông điệp `BOT ONLINE`, sau đó chờ lệnh. Khi nhận lệnh, bot chạy tiến trình con tương ứng:

- `http`: script dùng `requests.Session` với 30 luồng, gửi GET liên tục đến URL mục tiêu.
- `syn` / `udp`: gọi `hping3` với chế độ `--flood`.

Một chi tiết kỹ thuật đáng chú ý: `socket.create_connection(..., timeout=10)` giữ nguyên timeout cho cả lời gọi `recv()` sau đó. Nếu không gọi `sock.settimeout(None)`, bot sẽ ngắt kết nối sau 10 giây và liên tục kết nối lại, khiến danh sách bot trên C2 luôn rỗng tại thời điểm quan sát. Đoạn mã đúng:

```
sock = socket.create_connection((C2_IP, C2_PORT), timeout=10)

# create_connection's timeout also applies to recv(); clear it

# so the command channel blocks instead of reconnecting.

sock.settimeout(None)

sock.sendall(f"BOT ONLINE {socket.gethostname()}\n".encode())
```

3.5. Cấu hình phòng thủ trên máy nạn nhân

Suricata chạy trên `enp0s3` với bộ luật riêng:

```
alert http any any -> $HOME_NET 80 (msg:"ATT LAB HTTP FLOOD";
flow:established,to_server; threshold:type both, track by_src, count 50,
seconds 10; classtype:attempted-dos; sid:1000001; rev:1;)

alert tcp any any -> $HOME_NET 80 (msg:"ATT LAB SYN FLOOD"; flags:S;
threshold:type both, track by_src, count 100, seconds 5; classtype:attempted-
dos; sid:1000002; rev:1;)
```

```
alert tcp any any -> $HOME_NET 80 (msg:"ATT LAB SLOW CONNECTION FLOOD";
flags:S; threshold:type both, track by_src, count 40, seconds 60;
classtype:attempted-dos; sid:1000003; rev:1;)

alert http any any -> $HOME_NET any (msg:"ATT LAB LOIC USER-AGENT";
http.user_agent; content:"LOIC"; nocase; classtype:attempted-dos; sid:1000004;
rev:1;)

alert udp any any -> $HOME_NET any (msg:"ATT LAB UDP FLOOD"; threshold:type
both, track by_src, count 500, seconds 5; classtype:attempted-dos; sid:1000006;
rev:1;)
```

Các luật dùng `threshold:type both, track by_src` để chỉ cảnh báo khi một IP nguồn vượt ngưỡng trong khoảng thời gian xác định, tránh nhiễu từ lưu lượng bình thường.

Chặn tự động đọc `fast.log` và chèn luật UFW lên đầu danh sách:

```
tail -F "$LOG" | while IFS= read -r line; do

    if echo "$line" | grep -qE 'ATT LAB (HTTP|SYN|SLOW|LOIC|UDP)|FLOOD|LOIC';
then

        ip=$(echo "$line" | grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' | head -1)

        case " $WHITELIST " in

            *" $ip "*) continue ;;

        esac

        if ! grep -qx "$ip" "$SEEN"; then

            echo "$ip" >> "$SEEN"

            ufw insert 1 deny from "$ip" comment 'att-autoblock'

            echo "$(date -Is) BLOCK $ip" >> /var/log/att-blocks.log

        fi

    fi
```

```
done
```

Danh sách ngoại lệ gồm gateway `10.10.10.1`, victim `10.10.10.30`, backend `10.10.10.40` và `127.0.0.1`. Gateway được đưa vào danh sách trắng vì đường SSH từ WSL đi qua NAT gateway; nếu chặn gateway, phiên quản trị sẽ bị mất.

Nginx bảo vệ bổ sung giới hạn theo IP:

```
limit_req_zone $binary_remote_addr zone=req_per_ip:10m rate=10r/s;

limit_conn_zone $binary_remote_addr zone=conn_per_ip:10m;



server {

    listen 80 default_server;

    server_name _;



    limit_req zone=req_per_ip burst=20 nodelay;

    limit_conn conn_per_ip 20;



    if ($http_user_agent ~* (LOIC|Hping|slowloris)) {

        return 403;

    }



    location / {

        proxy_pass http://att_backend;

        proxy_next_upstream error timeout http_502 http_503 http_504;
```

```
    }

}
```

Với cấu hình này, mỗi IP chỉ được gửi trung bình 10 yêu cầu/giây (cho phép dồn tối đa 20 yêu cầu) và tối đa 20 kết nối đồng thời. Yêu cầu vượt ngưỡng bị trả mã 503.

3.6. Cấu hình cân bằng tải

```
upstream att_backend {

    server 127.0.0.1:8081 max_fails=1 fail_timeout=5s;

    server 10.10.10.40:8081 max_fails=1 fail_timeout=5s;

}
```

Backend-1 chạy ngay trên máy nạn nhân ở `127.0.0.1:8081`; backend-2 chạy trên VM riêng ở `10.10.10.40:8081`. Cả hai trả về JSON để dễ nhận biết backend nào đã phục vụ yêu cầu:

```
{"server":"backend-1","host":"att-victim"}

{"server":"backend-2","host":"att-backend2"}
```

---

4. Kết quả thực nghiệm

Mỗi bước đều có script thu thập minh chứng riêng, lưu vào thư mục `evidence/<timestamp>-<bước>/`. Số liệu dưới đây trích trực tiếp từ các tệp log đó.

4.1. Trạng thái ban đầu

- 5 VM nhận đúng IP tĩnh, SSH hoạt động.
- Victim: Nginx active, Suricata inactive, UFW inactive, auto-block inactive; Nginx đang dùng cấu hình baseline.
- Cân bằng tải hoạt động đúng: 6 yêu cầu liên tiếp lần lượt trả về backend-1 và backend-2 xen kẽ.

```
{"server":"backend-1","host":"att-victim"}

{"server":"backend-2","host":"att-backend2"}

{"server":"backend-1","host":"att-victim"}

{"server":"backend-2","host":"att-backend2"}
```

4.2. Bước 1 — Botnet tự tấn công

Sau khi cả hai bot đăng ký với C2, danh sách bot tại thời điểm kiểm tra:

```
["10.10.10.20", "10.10.10.21"]
```

Lệnh tấn công được gửi từ C2:

```
attack 10.10.10.30 80 25 http
```

C2 ghi nhận:

```
[15:46:39] ATTACK 10.10.10.30:80 type=http duration=25s -> 2 bot(s)
```

Cả hai bot nhận lệnh và chạy HTTP flood với 30 luồng:

```
ATT-Bot1: [15:46:40] attack -> 10.10.10.30:80 type=http duration=25s

ATT-Bot1: [FLOOD] http://10.10.10.30:80 threads=30 duration=25s

ATT-Bot1: [FLOOD] done sent=4427 errors=0



ATT-Bot2: [15:46:39] attack -> 10.10.10.30:80 type=http duration=25s

ATT-Bot2: [FLOOD] http://10.10.10.30:80 threads=30 duration=25s

ATT-Bot2: [FLOOD] done sent=4479 errors=0
```

Tổng cộng hai bot gửi thành công **8.906 yêu cầu** trong khoảng 25 giây. Trên máy nạn nhân ghi nhận **8.966 yêu cầu HTTP 200** (bao gồm cả lưu lượng kiểm thử trước đó) và

5.089 kết nối ở trạng thái TIME_WAIT, cho thấy số lượng kết nối ngắn hạn tăng mạnh:

```
Total: 157

TCP:   5095 (estab 1, closed 5089, orphaned 0, timewait 5089)



HTTP status counts:

   8966 200

total requests: 8966
```

Trong lúc tấn công, victim có 63 kết nối đang mở và 1.160 kết nối TIME_WAIT:

```
Total: 221

TCP:   1230 (estab 63, closed 1160, orphaned 1, timewait 1160)
```

Kết quả cho thấy botnet điều khiển được cả hai bot và tạo được tải đồng thời lên máy nạn nhân. Với quy mô nhỏ, dịch vụ vẫn trả lời được, nhưng số kết nối và tài nguyên xử lý tăng rõ rệt.

4.3. Bước 2 — Phòng chống

Script bật phòng thủ lần lượt chuyển Nginx sang cấu hình bảo vệ, khởi động Suricata trên `enp0s3`, bật UFW và chạy dịch vụ auto-block:

```
[1] Switch nginx to protected config

[2] Configure and start Suricata on enp0s3

suricata: active

[3] Enable UFW

Status: active

Default: deny (incoming), allow (outgoing), disabled (routed)
```

```
[4] Start auto-block

DEFENSE ENABLED
```

Tiếp tục cho hai bot tấn công, Suricata phát hiện các dấu hiệu flood và ghi vào `fast.log`:

```
11/08/2026-15:48:12.277345  [**] [1:1000001:1] ATT LAB HTTP FLOOD [**] {TCP}
10.10.10.20:39572 -> 10.10.10.30:80

11/08/2026-15:48:12.302675  [**] [1:1000001:1] ATT LAB HTTP FLOOD [**] {TCP}
10.10.10.21:48926 -> 10.10.10.30:80

11/08/2026-15:48:15.094191  [**] [1:1000003:1] ATT LAB SLOW CONNECTION FLOOD
[**] {TCP} 10.10.10.20:37892 -> 10.10.10.30:80

11/08/2026-15:48:19.671364  [**] [1:1000001:1] ATT LAB HTTP FLOOD [**] {TCP}
10.10.10.20:39572 -> 10.10.10.30:80

11/08/2026-15:48:22.231132  [**] [1:1000001:1] ATT LAB HTTP FLOOD [**] {TCP}
10.10.10.21:49024 -> 10.10.10.30:80

11/08/2026-15:48:28.970085  [**] [1:1000003:1] ATT LAB SLOW CONNECTION FLOOD
[**] {TCP} 10.10.10.21:56458 -> 10.10.10.30:80

11/08/2026-15:48:29.205909  [**] [1:1000002:1] ATT LAB SYN FLOOD [**] {TCP}
10.10.10.20:40774 -> 10.10.10.30:80
```

Chỉ 1–2 giây sau cảnh báo đầu tiên, auto-block chèn luật UFW cho cả hai IP:

```
2026-11-08T15:48:13+07:00 BLOCK 10.10.10.20

2026-11-08T15:48:14+07:00 BLOCK 10.10.10.21
```

Trạng thái UFW sau khi chặn (các luật DENY nằm ở đầu danh sách):

```
Status: active



     To                         Action      From
```

```
     --                         ------      ----

[ 1] Anywhere                   DENY IN     10.10.10.21               # att-
autoblock

[ 2] Anywhere                   DENY IN     10.10.10.20               # att-
autoblock

[ 3] 22/tcp                     ALLOW IN    Anywhere

[ 4] 80/tcp                     ALLOW IN    Anywhere
```

Kiểm chứng hiệu quả bằng hai phép thử:

- Từ bot đã bị chặn: `curl` không kết nối được, trả mã `000` sau 5 giây.
- Từ C2 (IP không tấn công): `curl` vẫn nhận `200` trong 4–6 ms.

```
curl from blocked bot:

000 5.001742s

CONNECTION FAILED (blocked)



curl from non-attacking C2:

200 0.004400s

200 0.005978s
```

Trong giai đoạn trước khi luật UFW kịp áp dụng, lớp giới hạn tần suất của Nginx đã trả 503 cho phần lớn lưu lượng. Access log cộng dồn sau bước 2 ghi nhận 9.212 response `200` và 3.070 response `503`; phần lớn yêu cầu tấn công bị chặn ở tầng ứng dụng, chỉ một phần nhỏ lọt qua ngưỡng burst 20.

Như vậy hệ thống phòng thủ hoạt động theo ba lớp: Suricata phát hiện, auto-block + UFW chặn ở tầng mạng, Nginx giới hạn tải ở tầng ứng dụng.

4.4. Bước 3 — Cân bằng tải và chịu lỗi

Trước khi kiểm thử, các luật chặn tự động được xóa nhưng UFW vẫn bật. Trạng thái

UFW trở về chỉ còn luật cho phép 22/80.

Phân phối bình thường: 10 yêu cầu liên tiếp cho kết quả 5/5:

```
{"server":"backend-1","host":"att-victim"}

{"server":"backend-2","host":"att-backend2"}

... (xen kẽ)

backend-1 responses: 5

backend-2 responses: 5
```

Kiểm tra chịu lỗi: dừng Nginx trên backend-2, gửi 6 yêu cầu, toàn bộ do backend-1 phục vụ:

```
backend-1 responses: 6

backend-2 responses: 0
```

Phục hồi: bật lại backend-2 và gửi 6 yêu cầu:

```
{"server":"backend-1","host":"att-victim"}

{"server":"backend-1","host":"att-victim"}

{"server":"backend-2","host":"att-backend2"}

{"server":"backend-1","host":"att-victim"}

{"server":"backend-2","host":"att-backend2"}

{"server":"backend-1","host":"att-victim"}



backend-1 responses: 4

backend-2 responses: 2
```

Hai yêu cầu đầu vẫn về backend-1 do backend-2 còn trong thời gian `fail_timeout=5s`, sau

đó Nginx đánh dấu backend-2 hoạt động trở lại và phân phối xen kẽ. Cơ chế round-robin và failover đều hoạt động đúng thiết kế.

4.5. Tổng hợp kết quả

Hạng mục	Kết quả đo được
Bot đăng ký C2	2 bot (`10.10.10.20`, `10.10.10.21`)
Lưu lượng bot gửi	8.906 yêu cầu trong 25 giây
Victim tiếp nhận	8.966 response HTTP 200, 5.089 TIME_WAIT
Cảnh báo Suricata	SID 1000001, 1000002, 1000003
Chặn tự động	2 IP bị UFW DENY sau 1–2 giây
Bot bị chặn	`000` sau 5 giây; C2 không bị chặn: `200`
Giới hạn tần suất	3.070 response 503
Cân bằng tải	5/5 khi bình thường, 6/0 khi mất backend-2, 4/2 khi phục hồi

---

5. Thảo luận

5.1. Đánh giá hiệu quả

- **Phát hiện:** Suricata nhận diện được cả HTTP flood và SYN flood nhờ luật ngưỡng theo IP nguồn. Việc dùng `threshold` giúp giảm cảnh báo giả so với việc bật cảnh báo cho từng gói tin.
- **Phản ứng:** kết hợp Suricata với UFW cho phép chặn tự động nhanh, không phụ thuộc vào thao tác thủ công. Đây là mô hình đơn giản nhưng hiệu quả của IDS + IPS thụ động.
- **Giảm tải:** `limit_req` và `limit_conn` bảo vệ backend khỏi bị quá tải bởi một IP, trong khi vẫn cho phép người dùng hợp lệ truy cập.
- **Sẵn sàng:** cân bằng tải giúp phân phối đều lưu lượng và tự động chuyển hướng khi backend gặp sự cố, giảm thời gian gián đoạn.

5.2. Hạn chế

- Quy mô lab nhỏ, chưa đại diện cho cường độ của một cuộc tấn công DDoS thực tế.
- Suricata chạy ở chế độ IDS, chỉ phát hiện chứ không chặn trực tiếp gói tin. Việc chặn phụ thuộc vào độ trễ đọc log của auto-block.
- UFW chặn toàn bộ lưu lượng từ một IP, nên nếu IP đó là NAT/proxy dùng chung, người dùng hợp lệ cũng bị ảnh hưởng.
- Giới hạn theo IP dễ bị né bằng cách tăng số lượng IP nguồn (tấn công phân tán thật sự).
- Nginx chỉ có một instance cân bằng tải, bản thân nó là điểm lỗi đơn (SPOF).
- Chưa có cơ chế chống giả mạo IP nguồn, chưa thử nghiệm slowloris quy mô lớn.

5.3. Hướng phát triển

- Chuyển Suricata sang chế độ IPS với NFQUEUE để chặn gói tin ngay khi phát hiện.
- Dùng `ipset`/`nftables set` thay cho danh sách luật UFW rời rạc, tăng hiệu năng khi số IP bị chặn lớn.
- Thêm HAProxy/Keepalived để loại bỏ điểm lỗi đơn ở tầng cân bằng tải.
- Bật SYN cookies, tinh chỉnh `somaxconn`, `tcp_max_syn_backlog` và giới hạn theo subnet thay vì từng IP.
- Mở rộng mô hình với nhiều bot hơn, mô phỏng phân tán theo subnet và đo lường bằng công cụ giám sát (Prometheus/Grafana).

---

6. Kết luận

Bài tập đã xây dựng thành công một lab ảo hóa 5 máy trên VirtualBox, điều khiển hoàn toàn từ WSL, mô phỏng đầy đủ vòng đời của một cuộc tấn công DDoS và các biện pháp ứng phó tương ứng:

1. **Tấn công:** botnet gồm C2 và hai bot nhận lệnh, gửi gần 9.000 yêu cầu trong 25 giây, làm tăng mạnh số kết nối trên máy nạn nhân.
2. **Phòng chống:** Suricata phát hiện flood theo ngưỡng, UFW chặn tự động hai IP nguồn trong 1–2 giây, Nginx giới hạn tần suất và trả 503 cho phần lớn lưu lượng vượt ngưỡng; người dùng hợp lệ từ IP khác vẫn truy cập bình thường.
3. **Cân bằng tải:** Nginx phân phối 5/5, chuyển toàn bộ lưu lượng sang backend-1 khi backend-2 dừng (6/0) và phân phối lại xen kẽ sau khi backend-2 phục hồi (4/2).

Kết quả cho thấy một hệ thống phòng thủ nhiều lớp kết hợp phát hiện, chặn và cân bằng tải có thể duy trì khả năng phục vụ ngay cả khi bị tấn công và khi một backend gặp sự cố. Toàn bộ mã nguồn, script và log minh chứng được lưu trong thư mục `vbox-lab` để có thể tái lập và kiểm chứng.

---

7. Tài liệu tham khảo

1. Suricata Documentation — https://docs.suricata.io/
2. Nginx — Module `ngx_http_upstream_module`: https://nginx.org/en/docs/http/ngx_http_upstream_module.html
3. Nginx — Module `ngx_http_limit_req_module`: https://nginx.org/en/docs/http/ngx_http_limit_req_module.html
4. Ubuntu Community Help — UFW: https://help.ubuntu.com/community/UFW
5. Oracle VirtualBox User Manual: https://www.virtualbox.org/manual/
6. cloud-init Documentation: https://cloudinit.readthedocs.io/
7. RFC 4987 — TCP SYN Flooding Attacks and Common Mitigations: https://www.rfc-editor.org/rfc/rfc4987

---

8. Phụ lục

Phụ lục A — Cấu trúc thư mục và lệnh chạy lại

```
vbox-lab/

├── config.sh                # IP, MAC, port, helper SSH

├── README.md                # hướng dẫn vận hành và xử lý lỗi

├── BAO-CAO-BTTL-ATTT.md     # báo cáo này

├── cloud-init/

│   ├── c2/user-data         # C2 server + control port 9998

│   ├── bot/user-data        # bot agent + HTTP flood
```

```
│   ├── victim/user-data      # Nginx LB, Suricata, UFW, auto-block
│   └── backend2/user-data    # backend-2 Nginx :8081
├── scripts/
│   ├── create_vms.sh         # tạo NAT network + ISO + 5 VM
│   ├── start_lab.sh          # boot và chờ cloud-init READY
│   ├── verify_baseline.sh    # kiểm tra IP, dịch vụ, LB baseline
│   ├── step1_attack.sh       # Bước 1
│   ├── step2_defense.sh      # Bước 2
│   ├── step3_loadbalance.sh  # Bước 3
│   ├── reset_lab.sh          # đưa victim về trạng thái chưa phòng thủ
│   └── collect_evidence.sh   # thu thập toàn bộ log/trạng thái
└── evidence/                 # log và ảnh minh chứng
```

Chạy lại toàn bộ:

```
cd vbox-lab

bash scripts/create_vms.sh

bash scripts/start_lab.sh

bash scripts/verify_baseline.sh

bash scripts/step1_attack.sh

bash scripts/step2_defense.sh

bash scripts/step3_loadbalance.sh

bash scripts/reset_lab.sh        # nếu muốn demo lại từ đầu
```

Phụ lục B — Ánh xạ yêu cầu bài tập với minh chứng

Yêu cầu	Minh chứng
Botnet có sẵn / mô phỏng	C2 + 2 bot; C2 log `ATTACK ... -> 2 bot(s)`; log bot nhận lệnh
Tự tấn công 2 máy	Hai bot cùng flood `10.10.10.30:80`; victim ghi 8.966 response 200
Công cụ phòng chống	`fast.log` (Suricata), `ufw status numbered`, `att-blocks.log`
Tường lửa	UFW `DENY IN 10.10.10.20/21 # att-autoblock`
Cân bằng tải	10 response xen kẽ backend-1/backend-2; cấu hình `upstream`
Chịu lỗi	Dừng backend-2: 6/0; bật lại: 4/2
Dùng WSL điều khiển VirtualBox	`VBoxManage.exe` gọi từ WSL; SSH port-forward 2222–2226

Các thư mục minh chứng tương ứng:

Bước	Thư mục
Baseline	`evidence/20261108-154610-baseline/`
Bước 1	`evidence/20261108-154632-step1-attack/`
Bước 2	`evidence/20261108-154750-step2-defense/`
Bước 3	`evidence/20261108-154902-step3-loadbalance/`
Toàn bộ trạng thái	`evidence/20261108-155453-full-state/`

Phụ lục C — Khó khăn và cách xử lý

Hiện tượng	Nguyên nhân và cách xử lý
SSH từ WSL bị `Connection refused`	WSL2 có loopback riêng; dùng Windows OpenSSH (`C:\Windows\System32\OpenSSH\ssh.exe`) để truy cập port-

Hiện tượng	Nguyên nhân và cách xử lý
	forward trên host
Windows OpenSSH báo `UNPROTECTED PRIVATE KEY FILE`	Siết quyền khóa bằng `icacls <key> /inheritance:r /grant:r "<user>:R"`
VM không có IP `10.10.10.x`	`network-config` dùng MAC dạng 12 ký tự liền; netplan yêu cầu dạng có dấu hai chấm
C2 trả về `bots` rỗng, bot online/offline liên tục	`socket.create_connection(timeout=10)` giữ timeout cho `recv()`; gọi `sock.settimeout(None)` sau khi kết nối
Suricata không chạy	Kiểm tra interface trong `override.conf` phải là `enp0s3`; xem `journalctl -u suricata`
Bot bị chặn nhưng SSH vẫn vào được	Thiết kế đúng: gateway `10.10.10.1` nằm trong danh sách trắng để không mất phiên quản trị
Round-robin chỉ trả về một backend	Dùng `curl` mỗi lần một kết nối mới; trình duyệt keep-alive sẽ dính một backend